SimpleMail, Documentation.

Group Details:

Member 1: Sumedh Jamadagni, 24CSB0A76

Member 2: Tarun Nagarajan 24CSB0A79

Purpose and architecture

This document covers the components and behavior required by the Computer Networking Programming Assignment. SimpleMail consists of `smserver`, `smspooler`, and `smclient`. The client and server use TCP; the server and spooler exchange complete messages through a shared-memory ring. Only the spooler creates and writes mail files. The authenticated server reads or deletes files for SMP mailbox operations.

The server uses nonblocking sockets and `poll()` to serve up to 128 clients on one configurable port. Each connection owns an input buffer, output buffer, protocol state, nonce, and bounded message staging record. Partial TCP reads are retained until a complete command or chunk arrives; partial writes remain queued. A waiting sender does not block other clients.

Build, startup and user registry

Build on Ubuntu/Linux with a C11 compiler, Make, POSIX shared memory, sockets and pthread support:

```
make clean
make
```

The Makefile builds all three programs with `-Wall -Wextra -Wpedantic` and links pthread/real-time support. Start the following in separate terminals, from the same project directory:

```
./smspooler users.txt
./smserver 2525 users.txt
./smclient 127.0.0.1 2525
```

Both daemons load the user registry at startup. Use one username/password pair per line, separated by a space. Usernames are ASCII letters, at most 20 characters, and are compared without case; mailbox paths use lowercase names. Passwords are case-sensitive ASCII alphanumeric strings, at most 30 characters. Duplicate usernames and invalid or missing registries cause startup failure. The parser also permits blank lines and comment lines beginning with #.

The sample users are `alice password1`, `bob secret2`, and `carol hunter7`. The implementation supports 128 registry entries. Passwords in this registry and the required DJB2 challenge mechanism follow the assignment's format.

Start the spooler before the server; the server waits up to five seconds for its shared object. A second live spooler is rejected. After a crash, a replacement spooler can remove a stale shared object whose owner is no longer alive. Stop the server before the spooler; the server must not continue using a queue without its consumer.

Shared-memory spool and synchronization

The POSIX object `/simplemail_v1_ring` contains eight fixed-capacity message slots. Each record holds the sender display name, accepted recipients and count, subject, body length, a 65,536-byte body buffer, and queue ticket. The queue header contains the owner PID, initialization magic, head/tail/count, ticket counter and a dequeue generation counter.

A process-shared pthread mutex protects record publication, indexes and counters. Process-shared `not_empty` and `not_full` conditions coordinate queue availability. These implement the assignment's permitted mutex/condition-variable alternative to unnamed semaphores. The spooler waits for records and releases queue capacity after dequeue. The frontend attempts insertion without waiting in the event loop; a completed message remains in its connection state until a slot is available.

The server assigns increasing queue tickets while the shared queue exists. The spooler validates each queued recipient and writes a copy to every accepted recipient. A fresh spooler run resets the queue ticket sequence; mailbox IDs are separate, durable counters. Queue contents are volatile, so `OK Queued` confirms acceptance into shared memory rather than completed disk persistence.

Mailbox files and IDs

At startup the spooler creates `mailboxes/` and one directory for every registered user. Each mail is a separate `<id>.txt` file. The stored format is:

```
From: <display name>
To: <comma-separated accepted recipients>
Subject: <subject or (no subject)>
Date: <YYYY-MM-DD HH:MM:SS>
---

<body>
```

Each mailbox's `.next-id` metadata starts at 1. The spooler reserves the next ID using a temporary metadata file, flushes and synchronizes it, renames it, and synchronizes the directory before creating the mail. A crash may skip an ID, but deletion and normal restart do not reuse it. A newly created empty mailbox can initialize metadata; an existing mailbox with missing or malformed metadata fails closed. The spooler does not reconstruct deletion history from remaining files.

Connection handshake and SMTP2

A new connection receives `WELCOME SimpleMail v1.0`. `MODE SEND` or `MODE RECV` receives `OK`; RECV then receives its authentication challenge. An invalid mode receives `ERR Unknown mode` and closes. A connection that sends no mode is closed after 30 seconds.

SMTP2 commands and responses end with CRLF. Command lines are limited to 512 bytes including CRLF. Sending requires no authentication; FROM carries a display name, rather than a registered account. The implementation permits 128-byte display names, up to 16 accepted recipients, and printable header text.

Command or state	Required behavior
FROM <display name>	Start a transaction; reply `OK Sender accepted.`
TO <username>	Accept registered recipients with `OK Recipient accepted.`; unknown users receive `ERR No such user.`
SUB <subject>	Require at least one accepted recipient; reply `OK Subject accepted.`; empty text is stored as `(no subject)`.
BODY_STREAM	Enter chunked body input; initially reply `OK Window 2048`.
CHUNK <byte_count>	Read exactly that many raw bytes after the header's CRLF; acknowledge the remaining window.
END_BODY	Queue the complete message, then reply `OK Queued <ticket> to N mailboxes`.
BODY	Reply `OK Send body, end with CRLF.CRLF.` and accept legacy dot-terminated input.
QUIT	Reply `BYE` and close after completing the transaction.

Out-of-sequence commands receive `ERR Bad sequence`. After a queued message, another FROM can start the next transaction on the same sending connection. Unknown recipients do not prevent subsequent valid TO commands. The client must receive at least one acceptance before proceeding to SUB.

In BODY_STREAM, byte counts frame the payload; raw data may contain line breaks and may arrive in several TCP reads. No extra separator is inserted between a chunk's raw bytes and the next command. In BODY, a line containing only `.` terminates input, including an empty body. A sender adds a dot to a body line that already starts with one; the server removes this extra dot. The combined body limit is 65,536 bytes in either mode; exceeding it returns `ERR Body too large.`.

Byte credits and backpressure

The byte window and the ring's message-slot count are different resources. Each complete chunk consumes credits and receives `OK Window <remaining>`. Consumed bytes move into the connection's bounded body record and become pending for credit restoration. A later `WINDOW_UPDATE <additional_credits>` restores those bytes when the spool queue has available capacity. This allows a first large upload to progress without an unrelated prior message.

When the ring is full, credit restoration waits. A sender that consumes its initial 2,048 bytes can therefore receive `OK Window 0` and must pause. The poll loop continues serving other clients. Once the spooler drains the ring, credit updates and waiting queue confirmations resume. Sending a chunk larger than the advertised balance, including any positive chunk at zero credits, returns `ERR Window overflow` and closes the connection. Each body remains bounded independently at 65,536 bytes.

SMP authentication and mailbox operations

After RECV selection, the server sends `AUTH REQUIRED <nonce>`, using eight random alphanumeric characters. The client hashes password followed by nonce, with no separator, using the assignment's DJB2 algorithm, and sends `AUTH <username> <unsigned-decimal hash>`. Successful authentication returns `OK Welcome <canonical username>`. Failed attempts receive `ERR Authentication failed`; the third receives `ERR Too many failures` and closes the connection.

Authenticated command	Response
LIST	`OK <count> messages`, then tab-separated ID, sender display name, subject and date rows; end with a single `.` line.
READ <id>	`OK`, followed by the mail headers/body, with dot-stuffing for body lines beginning with `.`; end with a single `.` line.
DELETE <id>	Remove that user's file and reply `OK Deleted.`; retain ID history.
COUNT	`OK <count>.`
QUIT	`BYE`, then close.
Unrecognized command	`ERR Unknown command.`

Missing READ/DELETE targets receive an error and leave the mailbox unchanged. Authentication fixes the mailbox identity for the session; IDs are interpreted within that user's directory. Client and server must use the same unsigned-long ABI width for DJB2. Command/header text is ASCII-compatible; streamed bodies are raw bytes. Dates and daemon logs use the Ubuntu host's local timezone.

Interactive client and event logs

`smclient <server_ip> <port>` provides the required numbered menu: 1 sends, 2 checks a mailbox, and 3 quits. Sending prompts for display name, recipients, subject and body; a blank recipient finishes the list, and a single-dot body line finishes terminal entry. The client handles chunks, credits and asynchronous updates. Mailbox commands provide list, read, delete, count and logout. Logout returns to the main menu using a fresh connection for the next mode. Retrieval removes transport dot-stuffing before displaying the body.

Both daemons write significant events to stdout with `[YYYY-MM-DD HH:MM:SS]` prefixes. Server events cover connection, mode, authentication, queue insertion, mailbox access and disconnection. Spooler events cover dequeue and per-recipient delivery or failure. These logs distinguish queue acceptance from later file delivery.

Verification and troubleshooting

The current source builds without warnings on Ubuntu 24.04.3 LTS, x86_64, GCC 13.3.0. Ten integration tests cover protocol, storage, concurrency and failure handling. Further scenario runs below force the assignment's boundary and recovery conditions, including an actual 30-second idle timeout, a stopped consumer, zero credits, and twenty simultaneous senders. The real interactive client is also exercised. Test scripts and capture drivers remain local; captured output is included in the documentation.

If the shared queue cannot be opened, start the spooler first. If binding fails, use a free port. For registry errors, correct the characters, lengths or duplicate name. For authentication failure, check password case and hash the password before the current nonce. For an unsafe ID counter, preserve the mailbox and restore valid metadata instead of resetting the number. For delayed sending with a full queue, inspect the spooler and delivery logs; queue confirmations and credit restoration wait for consumer capacity.

Ubuntu execution output

Build and regression results

Ubuntu | Build and regression results

```
Ubuntu 24.04.3 LTS
x86_64
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
$ make
Build completed without compiler warnings.
$ python3 tests/integration.py

----------------------------------------------------------------------
Ran 10 tests in 21.878s

OK
```

Registry errors and startup order

Ubuntu | Registry errors and startup order

```
$ ./smspooler missing.txt
smspooler: cannot open registry 'missing.txt': No such file or directory
exit status: 1
$ ./smserver 2525 missing.txt
smserver: cannot open registry 'missing.txt': No such file or directory
exit status: 1
$ ./smspooler bad.txt
smspooler: invalid registry entry on line 1
exit status: 1
$ ./smserver 2525 users.txt  # spooler is absent
smserver: cannot open spooler shared queue: No such file or directory
$ ./smspooler users.txt; ./smserver <port> users.txt
[2026-10-29 10:13:00] spooler: ready; 3 registered users, queue capacity 8
[2026-10-29 10:13:00] server: listening on port 42913 with 3 users
mailboxes/: alice, bob, carol

RESULT: PASS
```

Unknown mode and idle timeout

Ubuntu | Unknown mode and idle timeout

```
S> WELCOME SimpleMail v1.0
C> MODE INVALID
S> ERR Unknown mode
[connection closed by server]
S> WELCOME SimpleMail v1.0
[no MODE sent; waiting for the actual 30-second deadline]
[connection closed by server]
observed idle interval: 30.04 seconds

RESULT: PASS
```

Sequence and recipient validation

Ubuntu | Sequence and recipient validation

```
S> WELCOME SimpleMail v1.0
C> MODE SEND
S> OK
C> SUB too early
S> ERR Bad sequence
C> FROM Sumedh Jamadagni
S> OK Sender accepted.
C> SUB missing recipient
S> ERR Bad sequence
C> TO ghost
S> ERR No such user.
C> TO BOB
S> OK Recipient accepted.
C> TO carol
S> OK Recipient accepted.
C> SUB
S> OK Subject accepted.
C> BODY
S> OK Send body, end with CRLF.CRLF.
```

Default subject and dot stuffing

Ubuntu | Default subject and dot stuffing

```
C> READ 1
S> OK
S> From: Sumedh Jamadagni
S> To: bob,carol
S> Subject: (no subject)
S> Date: 2026-10-29 10:13:32
S> ---
S>
S> legacy text
S> ..leading dot
S> .
[later session: empty legacy body]
C> BODY
S> OK Send body, end with CRLF.CRLF.
C> .  [empty body]
S> OK Queued 2 to 1 mailboxes

RESULT: PASS
```

Authentication and mailbox errors

Ubuntu | Authentication and mailbox errors

```
C> MODE RECV
S> AUTH REQUIRED ffG02rlK
C> AUTH alice 11170462823096910224
S> ERR Authentication failed
C> AUTH alice 11170462823096910224
S> ERR Authentication failed
C> AUTH alice 11170462823096910224
S> ERR Too many failures
[connection closed by server]
C> MODE RECV
S> AUTH REQUIRED Lc36XifL
C> AUTH ALICE 9023256256581385876
S> OK Welcome alice
C> COUNT
S> OK 0.
C> LIST
S> OK 0 messages
S> .
C> READ 999
S> ERR No such message
C> DELETE 999
S> ERR No such message
C> PING
S> ERR Unknown command.
C> QUIT
S> BYE
RESULT: PASS
```

Body limit and window overrun

Ubuntu | Body limit and window overrun

```
[routine setup omitted; full transcript is provided]
C> BODY_STREAM
S> OK Window 2048
C> CHUNK 2048
C> [2048 raw body bytes]
S> OK Window 0
[31 further 2048-byte chunks sent; every acknowledgement checked]
C> END_BODY
S> WINDOW_UPDATE 2048
S> OK Queued 1 to 1 mailboxes
stored body byte count: 65536
C> BODY_STREAM
S> OK Window 2048
[65536 accepted body bytes; attempting one additional byte]
S> WINDOW_UPDATE 2048
C> CHUNK 1
S> ERR Body too large.
[connection closed by server]
C> BODY_STREAM
S> OK Window 2048
C> CHUNK 2049
S> ERR Window overflow
[connection closed by server]

RESULT: PASS
```

TCP fragments and line length

Ubuntu | TCP fragments and line length

```
[routine setup omitted; full transcript is provided]
C> TO bob  [sent one byte at a time]
S> OK Recipient accepted.
C> SUB [506-byte subject; complete command is 512 bytes]
S> OK Subject accepted.
C> BODY_STREAM
S> OK Window 2048
C> CHUNK 21
C> [21 raw body bytes]
S> OK Window 2027
C> END_BODY
S> WINDOW_UPDATE 21
S> OK Queued 1 to 1 mailboxes
C> SUB [507-byte subject; complete command is 513 bytes]
S> ERR Invalid or overlong command line
[connection closed by server]

RESULT: PASS
```

Full queue, zero window and recovery

Ubuntu | Full queue, zero window and recovery

```
[routine setup omitted; full transcript is provided]
$ kill -STOP <spooler-pid>  # controlled pause
fill slot 1: OK Queued 1 to 1 mailboxes
[six intermediate filled-slot confirmations omitted]
fill slot 8: OK Queued 8 to 1 mailboxes
C> BODY_STREAM
S> OK Window 2048
C> CHUNK 2048
C> [2048 raw body bytes]
S> OK Window 0
[window is zero; no credit update while ring is full]
ninth END_BODY: no queue confirmation while all 8 slots are full
unrelated authenticated COUNT while blocked: 0
[intermediate exchanges omitted; see full transcript]
S> ERR Window overflow
[connection closed by server]
$ kill -CONT <spooler-pid>
waiting sender: OK Queued 9 to 1 mailboxes
S> WINDOW_UPDATE 2048
C> CHUNK 6
C> [6 raw body bytes]
S> OK Window 2042
C> END_BODY
S> WINDOW_UPDATE 6
S> OK Queued 10 to 1 mailboxes
delivered mailbox count after recovery: 10

RESULT: PASS
```

Twenty concurrent senders

Ubuntu | Twenty concurrent senders

```
sender 01: OK Queued 10 to 1 mailboxes
sender 02: OK Queued 11 to 1 mailboxes
sender 03: OK Queued 12 to 1 mailboxes
[14 other successful sender confirmations omitted]
sender 18: OK Queued 7 to 1 mailboxes
sender 19: OK Queued 6 to 1 mailboxes
sender 20: OK Queued 2 to 1 mailboxes
20 simultaneous senders; 20 unique tickets; mailbox COUNT = 20
elapsed: 2.981 seconds

RESULT: PASS
```

IDs after deletion and restart

Ubuntu | IDs after deletion and restart

```
[routine setup omitted; full transcript is provided]
C> BODY_STREAM
S> OK Window 2048
C> CHUNK 15
C> [15 raw body bytes]
S> OK Window 2033
C> END_BODY
S> WINDOW_UPDATE 15
S> OK Queued 1 to 1 mailboxes
C> DELETE 1
S> OK Deleted.
C> COUNT
S> OK 0.
before restart .next-id: 2
$ restart smspooler and smserver
C> BODY_STREAM
S> OK Window 2048
C> CHUNK 15
C> [15 raw body bytes]
S> OK Window 2033
C> END_BODY
S> WINDOW_UPDATE 15
S> OK Queued 1 to 1 mailboxes
C> LIST
S> OK 1 messages
S> 2  Tarun Nagarajan   Demonstration  2026-10-29 10:13:47
S> .
mailbox files: 2.txt
mailbox ID 1 not reused; queue ticket sequence is scoped to a spooler run

RESULT: PASS
```

Interactive send menu

Ubuntu | Interactive send menu

```
$ ./smclient 127.0.0.1 <port>
1. Send a mail
2. Check my mailbox
3. Quit
Choice: 1
WELCOME SimpleMail v1.0
OK
Sender display name: Tarun Nagarajan
OK Sender accepted.
Recipient username (blank when finished): bob
OK Recipient accepted.
Recipient username (blank when finished):
Subject (blank for no subject): Client menu test
OK Subject accepted.
Enter the body one line at a time. A line containing only a single period ends it.
Client output from Ubuntu
.
OK Window 2048
OK Window 2021
Server credit update: WINDOW_UPDATE 27
OK Queued 1 to 1 mailboxes
1. Send a mail
2. Check my mailbox
3. Quit
Choice: 3
```

Interactive mailbox menu

Ubuntu | Interactive mailbox menu

```
$ ./smclient 127.0.0.1 <port>
Choice: 2
WELCOME SimpleMail v1.0
Mailbox nonce: ecZxr0g4
Username: bob
Password: secret2
OK Welcome bob
Mailbox: [L]ist, [R]ead, [D]elete, [C]ount, [Q]uit
Choice: L
OK 1 messages
1  Tarun Nagarajan   Client menu test  2026-10-29 10:21:16
.
Choice: R
Message ID: 1
OK
From: Tarun Nagarajan
To: bob
Subject: Client menu test
Date: 2026-10-29 10:21:16
---
Client output from Ubuntu
Choice: D
Message ID: 1
OK Deleted.
Choice: C
OK 0.
Choice: Q
BYE
RESULT: PASS
```

Timestamped server and spooler events

Ubuntu | Timestamped server and spooler events

```
[2026-10-29 10:21:16] server: listening on port 57043 with 3 users
[2026-10-29 10:21:16] server: client connected
[2026-10-29 10:21:16] server: selected SEND mode
[2026-10-29 10:21:16] server: queued ticket 1 from Tarun Nagarajan to 1 mailboxes
[2026-10-29 10:21:16] server: client disconnected
[2026-10-29 10:21:16] server: client connected
[2026-10-29 10:21:16] server: selected RECV mode
[2026-10-29 10:21:16] server: authenticated mailbox bob
[2026-10-29 10:21:16] server: mailbox bob counted
[2026-10-29 10:21:16] server: client disconnected
[2026-10-29 10:21:16] server: client connected
[2026-10-29 10:21:16] server: selected RECV mode
[2026-10-29 10:21:16] server: authenticated mailbox bob
[2026-10-29 10:21:16] server: mailbox bob listed
[2026-10-29 10:21:16] server: mailbox bob read message 1
[2026-10-29 10:21:16] server: mailbox bob deleted message 1
[2026-10-29 10:21:16] server: mailbox bob counted
[2026-10-29 10:21:16] server: mailbox bob logged out
[2026-10-29 10:21:16] server: client disconnected
[2026-10-29 10:21:16] server: stopping
[2026-10-29 10:21:16] spooler: ready; 3 registered users, queue capacity 8
[2026-10-29 10:21:16] spooler: ticket 1 dequeued (1 recipients)
[2026-10-29 10:21:16] spooler: ticket 1 delivered to bob
[2026-10-29 10:21:17] spooler: stopping

RESULT: PASS
```